

Instituto Tecnológico de Costa Rica

EL5859 Computación Heterogénea: II Semestre 2026

Profesor: Luis G. León-Vega, Ph.D

Evaluación: Laboratorio Semana 9

Este es el laboratorio práctico para estudiar aceleración heterogénea con GPU usando CUDA. El punto de partida es una simulación de la caída de una gota sobre un estanque, implementada como una ecuación de onda 2D amortiguada. El programa genera un video en escala de grises donde se observa la propagación de las ondas producidas por la gota.

El objetivo no es solamente portar el programa a GPU, sino seguir un proceso de ingeniería basado en evidencia: comprender el código, medirlo, identificar las funciones críticas, acelerar dichas funciones, validar que el resultado sea correcto y justificar cada optimización realizada.

Para este laboratorio, necesitará:

- Una computadora.
- Una NVIDIA Jetson Nano.
- Sistema Operativo Linux con su entorno de desarrollo.
- Compilador de C/C++.
- CUDA Toolkit (en la Jetson).
- Meson y Ninja.
- OpenCV.
- Nsight Systems, Nsight Compute o nvprof, según disponibilidad (en la Jetson).
- El código base en el directorio `drop-simulation/`.

Este laboratorio está diseñado para ejecutarse en grupos personas. Cada integrante debe participar en la comprensión, medición, implementación y análisis del código.

Objetivo

Este laboratorio tiene como objetivo comprender, mediante un ejercicio práctico, cómo acelerar una aplicación numérica usando GPU y cómo evaluar el impacto real de distintas optimizaciones. Al finalizar, cada grupo debe poder explicar qué partes del programa conviene portar a GPU, cómo validar los resultados y por qué una optimización mejora o no mejora el rendimiento.

Preparación del entorno

Verifique que tiene instalado el compilador:

```
g++ --version
```

Verifique que tiene instalado Meson:

```
meson --version
ninja --version
```

Verifique que OpenCV está disponible:

```
pkg-config --modversion opencv4
```

Verifique que CUDA está disponible:

```
nvcc --version
nvidia-smi
```

Verifique las herramientas de perfilado disponibles:

```
nsys --version
nvprof --version
```

Si alguna herramienta no está disponible, indique la limitación en el reporte y utilice una herramienta equivalente.

Lectura previa

Antes de iniciar la implementación, lea la sección de optimización en GPU del libro del curso. Ponga especial atención a los conceptos relacionados con jerarquía de memoria, localidad de datos, paralelismo, representación numérica y costo de comunicación entre componentes del sistema.

En el reporte, incluya una breve discusión donde relacione al menos tres ideas de la lectura con las decisiones tomadas durante el laboratorio. Por ejemplo:

- Por qué una función fue seleccionada para optimización.
- Cómo afecta el acceso a memoria al rendimiento de los kernels.
- Qué compromiso existe entre precisión numérica y velocidad.
- Por qué reducir comunicación host-device puede mejorar el tiempo total aunque el kernel individual no cambie.

Ejercicio A: construcción y ejecución del código base

Ingresa al directorio del proyecto:

```
cd drop-simulation
```

Construya el código base:

```
meson setup build
meson compile -C build
```

Ejecute el programa:

```
./build/drop_simulation
```

El programa debe generar el archivo:

```
output/drop_simulation.mp4
```

Registre:

- Tiempo total de ejecución.
- Cantidad de pasos simulados.
- Rendimiento reportado en pasos por segundo.
- Resolución, duración y FPS usados por el programa.

Ejercicio B: comprensión del código

Lea el archivo principal del programa. Comente la funcionalidad de cada función en el código fuente. Los comentarios deben explicar el propósito de la función y su papel dentro de la simulación.

Como mínimo, identifique y explique:

- La estructura de configuración de la simulación.
- La función que inicializa la perturbación de la gota.
- La función que calcula la absorción en los bordes.
- La función que actualiza la ecuación de onda.
- La función que renderiza cada cuadro del video.
- La función principal y el ciclo de simulación.

Elabore un diagrama de flujo del programa. El diagrama debe mostrar, al menos:

- Inicialización de parámetros.
- Reserva de memoria.
- Perturbación inicial de la gota.
- Ciclo de simulación.
- Actualización de la malla.
- Renderizado del cuadro.
- Escritura del video.
- Cálculo de métricas de rendimiento.

Explique con sus propias palabras qué hace el programa y qué modelo físico aproxima.

Ejercicio C: perfilado del código en CPU

Perfilar el programa base antes de modificarlo. El objetivo es determinar cuáles funciones consumen la mayor cantidad de tiempo.

Use al menos dos de las siguientes herramientas:

- perf
- Valgrind Callgrind
- Google Performance Tools
- Instrumentación manual con temporizadores

Determine:

- Cuáles funciones son las más críticas.
- Qué porcentaje aproximado del tiempo consume cada una.
- Cuál función conviene portar primero a GPU.
- Cuáles funciones no conviene portar y por qué.

Ejercicio D: porteo de funciones críticas a GPU

Portee a CUDA las funciones más críticas identificadas en el perfilado. Debe justificar por qué seleccionó esas funciones.

Como punto de partida, se espera que el grupo considere portar:

- La actualización de la ecuación de onda.
- El cálculo usado para renderizar cada cuadro.

El porteo debe incluir:

- Reserva de memoria en GPU.
- Copia de datos entre host y device.
- Kernels CUDA para las funciones seleccionadas.
- Configuración de bloques e hilos.
- Manejo correcto de bordes de la malla.
- Integración con el flujo existente de generación del video.

Mida el rendimiento de esta primera versión GPU sin aplicar optimizaciones adicionales. Esta versión será la línea base GPU.

Ejercicio E: validación de resultados

Antes de optimizar, valide que la versión GPU produce resultados equivalentes a la versión CPU.

La validación debe incluir al menos una de las siguientes estrategias:

- Comparación numérica entre CPU y GPU para la malla de alturas.
- Error absoluto medio.
- Error absoluto máximo.
- Comparación visual de algunos cuadros generados.
- Explicación de cualquier diferencia observada.

Use tolerancias razonables para punto flotante. Si cambia la representación numérica durante las optimizaciones, debe repetir la validación.

Ejercicio F: optimización de la versión GPU

Optimice la versión GPU aplicando al menos tres técnicas distintas. Cada etapa de optimización debe medirse por separado usando Nsight Systems, Nsight Compute o nvprof.

Puede considerar las siguientes técnicas:

- Acceso coalesced a memoria global.
- Uso de memoria compartida para cargar vecindarios de la malla.
- Reducción de comunicación host-device.
- Ejecución de varios pasos de simulación antes de copiar datos al host.
- Uso de streams y copias asíncronas.
- Uso de `-use_fast_math`.
- Representación numérica alternativa, por ejemplo float, half o almacenamiento de menor precisión.
- Ajuste del tamaño de bloques e hilos.

Para cada optimización, mida como mínimo:

- Tiempo total de ejecución.
- Tiempo de kernels.
- Tiempo de copias host-device y device-host.
- Throughput o pasos por segundo.
- Speedup respecto a CPU.

- Speedup respecto a la versión GPU anterior.
- Error numérico respecto a la versión CPU.

Ejemplo con Nsight Systems:

```
nsys profile -o profile_baseline ./build/drop_simulation
```

Ejemplo con nvprof, si está disponible:

```
nvprof ./build/drop_simulation
```

Ejercicio G: reporte de optimizaciones

El grupo debe elaborar un reporte que explique el proceso seguido y los resultados obtenidos. El reporte debe incluir:

- Descripción del programa base.
- Diagrama de flujo.
- Explicación de cada función del programa.
- Resultados del perfilado CPU.
- Justificación de las funciones portadas a GPU.
- Diseño de los kernels CUDA.
- Estrategia de validación CPU vs GPU.
- Resultados de cada optimización.
- Capturas o salidas relevantes de Nsight o nvprof.
- Análisis de rendimiento.
- Conclusiones.

Incluya una tabla similar a la siguiente:

Versión	Tiempo total	Pasos/s	Speedup	Error máximo
CPU base			1.00x	
GPU base				
Optimización 1				
Optimización 2				
Optimización 3				

Para cada optimización, explique:

- Qué problema intenta resolver.

- Qué cambio se realizó en el código.
- Qué evidencia de perfilado justifica el cambio.
- Cuánto mejoró o empeoró el rendimiento.
- Qué impacto tuvo sobre la precisión numérica.

Entregable

Debe entregarse un repositorio de Git con:

- Código CPU original comentado.
- Código CUDA funcional.
- Al menos tres versiones u optimizaciones documentadas de forma incremental.
- Instrucciones de compilación y ejecución.
- Resultados de perfilado.
- Evidencia de validación CPU vs GPU.
- Video generado por la versión final.
- Reporte.

El enlace del repositorio debe subirse a través del TecDigital.

Rúbrica

La calificación total del laboratorio es de 100 puntos, distribuidos de la siguiente forma:

Criterio	Descripción	Puntaje
Seguimiento de instrucciones	Entrega todos los archivos solicitados, ejecuta las herramientas indicadas, incluye los comandos usados y respeta la estructura del laboratorio.	15 pts
Calidad del reporte	Presenta resultados completos, tablas claras, evidencia suficiente de las herramientas utilizadas y datos reproducibles para cada experimento.	20 pts
Análisis del reporte	Interpreta correctamente los resultados, justifica las decisiones basadas en el <i>profiling</i> y explica las diferencias.	20 pts
Propuesta de optimización y su justificación	Propone una optimización coherente con la evidencia, explica las hipótesis, compara antes y después, y concluye si el cambio mejoró el rendimiento.	30 pts
Presentación del reporte	El documento está ordenado, bien redactado, sin errores importantes de formato, con figuras o tablas legibles y conclusiones claras.	15 pts
Total		100 pts

Para cada criterio se puede asignar puntaje parcial según el nivel de cumplimiento: completo, mayormente completo, incompleto o ausente.

Asimismo, se recuerda completar la cross-evaluación y se aplicará de ahora en adelante la norma completa sobre el uso de IA.